

“DATA SCIENCE TO WIZINOVA”

**INTERNSHIP REPORT SUBMITTED TO THE BHARATHIYAR
UNIVERSITY FOR THE AWARD OF THE DEGREE OF
MASTER OF BUSINESS ADMINISTRATION**

By

GAYATHRI P

Reg. No. 1P23MB009

Under the Guidance of

Dr. Suganya S

MCA & Ph.D.

Associate Professor



SCHOOL OF MANAGEMENT STUDIES - PG

RVS COLLEGE OF ARTS AND SCIENCE (AUTONOMOUS)

Affiliated to Bharathiyar University, Approved by AICTE

Re Accredited with 'A+' Grade by NAAC

Sulur, Coimbatore – 641 402.

NOVEMBER 2024

CERTIFICATE

This is to certify that the Internship report, entitled “DATA SCIENCE TO WIZINOVA”, submitted to the Bharathiyar University, in partial fulfilment of the requirements for the award of the DEGREE OF MASTER OF BUSINESS ADMINISTRATION, is a record of original work done by Ms. GAYATHRI P, during the period 16-05-2024 TO 29-06-2024 of her internship in school of Management Studies - PG, RVS College of Arts and Science (Autonomous), Coimbatore - 641402, under my supervision and guidance and the internship report has not formed the basis for the award of any Degree / Diploma / Associateship / Fellowship or other similar title of any candidate of any University.

Date:

Director

Signature of the Guide

Date of Viva-voce Examination held on _____.

Internal Examiner

External Examine

DECLARATION

The internship project entitled “Data Science to Wizinova” is submitted to Bharathiyar University in partial fulfillment of the requirements for the award of the Master of Business Administration degree. The work presented is the result of original and independent research conducted from 16-05-2024 to 29-06-2024 under the supervision and guidance of Dr. Suganya S, Associate Professor, School of Computer Studies-PG, RVS College of Arts and Science (Autonomous), Coimbatore – 641 402. This research has not been used as the basis for the award of any other degree, diploma, associateship, fellowship, or similar title at any other university.

Date:

Signature of the Candidate

ACKNOWLEDGEMENT

Deepest gratitude and appreciation are extended to everyone who contributed to the success of the internship at Wizinova and the completion of this project report. This journey has been both enlightening and enriching, and it was made possible through the support, guidance, and encouragement of several individuals and the organization as a whole.

Profound thanks are due to Dr. Thulasivelu K, Director of RVS School of Management Studies, Sulur, Coimbatore, for his unwavering support, encouragement, and gracious permission, which were instrumental in facilitating the start and completion of this internship journey.

Sincere appreciation is extended to Wizinova for providing the opportunity to contribute to the firm. The experience of working with such a dynamic and innovative company has been invaluable, greatly enhancing professional skills and knowledge. The resources and support provided throughout the internship enabled substantial learning, growth, and the achievement of goals.

Special thanks are also owed to Mr. Karthikeyan, HR of Wizinova, for his unwavering guidance, support, and encouragement. His patience in clarifying complex concepts, readiness to share knowledge, and constructive feedback were crucial in developing a deeper understanding of the technology and operations at Wizinova.

Gratitude is also expressed to Dr. S. Suganya, Associate Professor of the School of Computer Studies at RVS Arts and Science, for her steadfast support and guidance during the internship. Her leadership, mentorship, and belief in the potential of the intern served as a constant source of motivation and inspiration, greatly contributing to professional growth and development.

CERTIFICATE OF INTERNSHIP



This is presented to

Gayathri P

Have completed her **45 Days** internship program on **Data Science**
from the period 16-05-2024 to 29-06-2024



Naaga Nivethitha
Naaga Nivethitha
CEO

CONTENTS

Chapter No.	Title	Page No.
I	Company and Business Understanding	
II	Data Understanding	
III	Data Insights	
IV	Learning Outcomes, Challenges Faced and Recommendations	
	Annexure 1.1. Attendance 1.2. Work sheet/Work Diary	

COMPANY AND BUSINESS UNDERSTANDING

Wizino, established in 2021, is a pioneering software development company offering a wide range of innovative and customized solutions for businesses across various industries. Since its inception, the company has focused on addressing the unique needs of its clients, helping them streamline their operations and drive growth through technology.

At the core of Wizino's expertise are its Customer Relationship Management (CRM) and Enterprise Resource Planning (ERP) solutions, designed to enhance business efficiency and improve customer engagement. The company's tailored CRM systems enable businesses to manage their customer interactions more effectively, while its ERP solutions integrate essential processes across departments, improving operational flow.

Wizino also offers cutting-edge digital marketing services, helping businesses expand their online presence, attract new customers, and improve brand visibility. Additionally, the company excels in the development of web and mobile applications, providing scalable and user-friendly platforms for businesses to engage with their audiences.

Committed to excellence and future-ready solutions, Wizino constantly adapts to the latest industry trends and emerging technologies. By delivering comprehensive tools and services, the company empowers its clients to remain competitive, agile, and prepared for the ever-evolving demands of the digital landscape.

The company is a dynamic provider of digital marketing and development services, focusing on delivering comprehensive and targeted solutions to clients. With a broad service portfolio, the company is well-equipped to handle various aspects of digital and online presence.

Services Offered:

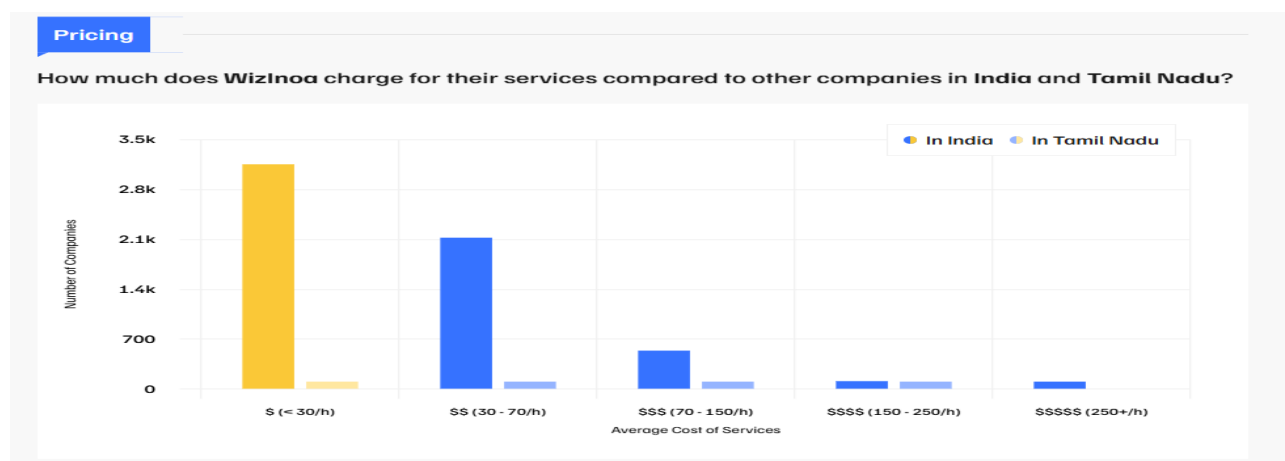
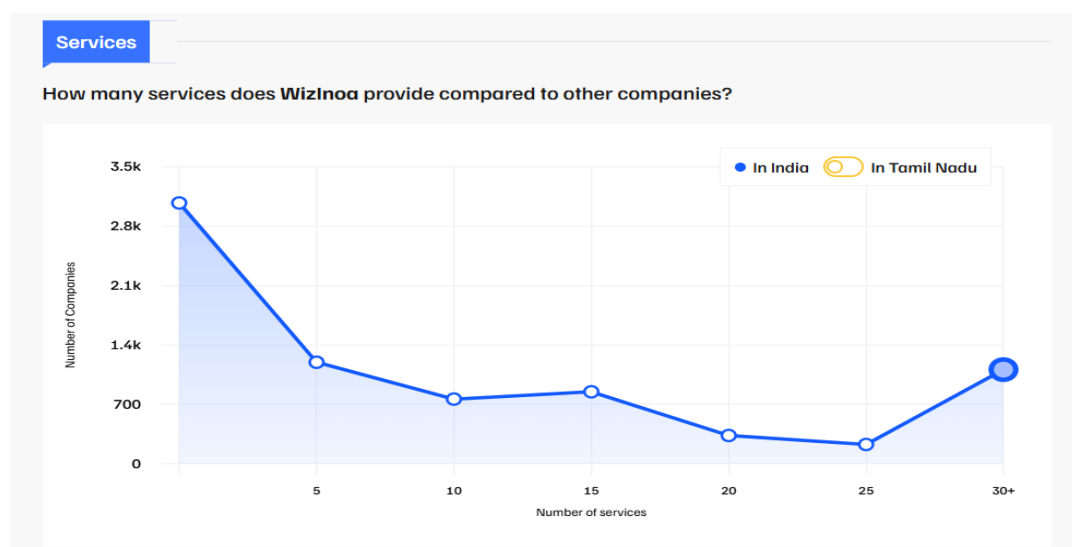
1. **Mobile App Development:** Building custom mobile applications to enhance user engagement.
2. **Web Development:** Creating robust, responsive websites tailored to client needs.
3. **UX/UI Design:** Designing user-centric interfaces to improve user experience.
4. **Web Design:** Crafting visually appealing and functional web layouts.
5. **SEO (Search Engine Optimization):** Optimizing content to rank higher in search engines.
6. **Pay Per Click (PPC):** Managing PPC campaigns to maximize advertising ROI.
7. **Digital Marketing:** Implementing multi-channel digital strategies for brand growth.
8. **Other Marketing Services:** Offering niche marketing solutions based on client requirements.
9. **Content Marketing:** Developing high-quality content to engage audiences.
10. **Email Marketing:** Crafting and distributing personalized email campaigns.
11. **Advertising:** Managing ad campaigns across various platforms.
12. **Social Media Marketing (SMM):** Boosting brand visibility through social media.
13. **Marketing Strategy:** Developing data-driven strategies for business growth.
14. **Public Relations (PR):** Enhancing brand image through effective PR campaigns.

15. **Branding:** Creating memorable brand identities.

16. **Mobile & App Marketing:** Promoting apps and mobile content to increase reach.

17. **Media Planning & Buying:** Strategizing and buying media space for maximum impact.

18. **Market Research:** Conducting research to understand market trends and customer needs.



Service Comparison: WizInoa stands out in India and Tamil Nadu by offering a wide range of services. While most companies provide fewer than 10 services, WizInoa is among the few offering 30+ services, making it a comprehensive provider.

Market Position: WizInoa's pricing is competitive and aligns with industry standards, making it an accessible and preferred choice for clients in both India and Tamil Nadu.

CHAPTER -II

Data Understanding

* Size

DATA SET	ROW	COLUMNS
TATA GROUPS	115	7
ADANI GROUPS	35	7
RELIANCE	35	7

i) Variables Type

DATA SET	VARIABLES	TYPE
TATA GROUPS	Date	Date
	Open	Double
	High	Double
	Low	Double
	Close	Double
	Adjust Close	Double
	Volume	Integer
ADANI GROUPS	Date	Date
	Open	Float
	High	Float
	Low	Float
	Close	Float
	Adjust Close	Float
	Volume	Integer
RELIANCE	Date	Date
	Open	Float
	High	Float
	Low	Float
	Close	Float
	Adjust Close	Float
	Volume	Integer

DATA QUALITY ASSESSMENT

TOOL USED: MY SQL WORKBRENCH.

PROBLEM STATEMENT:

The task involves identifying and addressing any missing or invalid values within the dataset, which can skew financial insights. The problem statement focuses on cleaning the dataset and maintaining its accuracy.

STEP 1: checking the missing values.

The SQL query used in this step is designed to ensure that none of the key columns in the dataset have null or missing values.

SQL Query:

```
SELECT * FROM table_name  
WHERE Date IS NOT NULL  
AND Open IS NOT NULL  
AND High IS NOT NULL  
AND Low IS NOT NULL  
AND AdjClose IS NOT NULL  
AND Volume IS NOT NULL;
```

This query retrieves all rows from the "Reliance" ,"Adanigroups","Tata" table where none of the following columns have missing values:

- Date: The trading date.
- Open: The opening price of the stock.
- High: The highest price recorded.
- Low: The lowest price recorded.
- AdjClose: The adjusted closing price.
- Volume: The number of shares traded.

```

6 • SELECT * FROM reliance
7 WHERE Date IS NOT NULL
8 AND Open IS NOT NULL
9 AND High IS NOT NULL
10 AND Low IS NOT NULL
11 AND AdjClose IS NOT NULL
12 AND Volume IS NOT NULL;

```

Result Grid							
		Filter Rows:		Export:		Wrap Cell Content	
	Date	Open	High	Low	Close	AdjClose	Volume
1	2010-01-04	197	197.65	192.6	193.5	193.5	18350
2	2010-01-05	191	198.5	190.5	194	194	57941
3	2010-01-06	195.5	196.4	190.7	191.9	191.9	14784
4	2010-01-07	190.5	193.8	183.25	187.1	187.1	21959
5	2010-01-08	189.6	192	185.15	186.95	186.95	30950
6	2011-01-03	33.59	34.2	33.5	33.59	33.59	111565
7	2011-01-04	33.9	33.9	33.01	33.37	33.37	63695
8	2011-01-05	33	33.59	32.61	32.92	32.92	16390
9	2011-01-06	33.39	33.39	31.98	32.13	32.13	24735
10	2011-01-07	32.2	32.59	30.8	30.99	30.99	147190

```

7 WHERE Date IS NOT NULL
8 AND Open IS NOT NULL
9 AND High IS NOT NULL
10 AND Low IS NOT NULL
11 AND AdjClose IS NOT NULL
12 AND Volume IS NOT NULL;

```

Result Grid							
		Filter Rows:		Export:		Wrap Cell Content	
	Date	Open	High	Low	Close	AdjClose	Volume
1	2003-01-01	1.09668	1.11508	1.09668	1.10664	0.033884	404318
2	2003-01-02	1.10818	1.11124	1.10511	1.10971	0.033978	407763
3	2003-01-03	1.11891	1.12581	1.10435	1.11048	0.034002	410308
4	2003-01-06	1.11891	1.11891	1.09745	1.10281	0.033767	404762
5	2003-01-07	1.10358	1.11124	1.09438	1.10511	0.033837	407737
6	2019-07-18	48.3	49.75	46.95	47.5	47.5	938262
7	2019-07-19	48	48.15	45.2	45.8	45.8	1180434
8	2019-07-22	45.5	47.45	44.7	46.8	46.8	1022220
9	2019-07-23	46.9	49.35	46.7	48.8	48.8	1756709
10	2019-07-24	49.4	49.5	48.25	48.45	48.45	671080
11	2008-01-01	255.8	263.8	252.74	259.37	238.616	2456740
12	2008-01-02	262.05	264.8	254	261.49	240.566	3519075
13	2008-01-03	261	262.94	252.23	255.16	234.743	2525290
14	2008-01-04	255.76	258.96	246.6	248.07	228.22	2753975
15	2008-01-07	248	255.78	243	245.45	225.81	3154815

```

SELECT * FROM tata
WHERE Date IS NOT NULL
AND Open IS NOT NULL
AND High IS NOT NULL
AND Low IS NOT NULL
AND Volume IS NOT NULL;

```

Date	Open	High	Low	Close	Adj Close	Volume
2003-01-01	15.940852	15.940852	15.940852	15.940852	13.515811	0
2003-01-02	21.539383	21.539383	20.086329	21.368435	18.117706	496
2003-01-03	21.069277	21.069277	21.069277	21.069277	17.864059	116
2003-01-06	20.941065	20.941065	20.941065	20.941065	17.755354	467
2003-01-07	20.941065	20.941065	20.941065	20.941065	17.755354	0
2003-01-01	17.697426	17.875242	17.519608	17.585119	14.506817	165908
2003-01-02	17.688066	17.734859	17.547686	17.575762	14.499096	59281
2003-01-03	17.229486	17.744219	17.229486	17.65999	14.568581	58330
2003-01-06	17.416662	17.875242	17.416662	17.678707	14.584023	231504
2003-01-07	17.781654	17.875242	17.500891	17.557043	14.483654	167618
2003-01-01	33.5	36.299999	33.5	34.799999	32.149387	382306
2003-01-02	35	35.5	33.049999	33.25	30.717447	77357
2003-01-03	34	34	32.650002	32.849998	30.347914	49463
2003-01-06	33	33.200001	31.549999	31.950001	29.516466	55208

Result :

The result shows a subset of rows from the dataset where all values are present. This indicates that no missing values were found in the queried dataset for these columns.

STEP 2: Check for Duplicates in Date:

Group by Date: The SQL query groups all rows based on the date value, meaning all records that share the same date are treated as one group.

Count Occurrences: For each date, the query counts how many records exist. If there is more than one record for a specific date, it suggests duplicate entries.

Filter Duplicates: The query filters the results to show only those dates where more than one record exists (i.e., duplicates). This is done using the HAVING clause to limit the result to groups with a count greater than 1.

```

18 • SELECT Date, COUNT(*)
19 FROM tata
20 GROUP BY Date
21 HAVING COUNT(*) > 1;

```

Result Grid   Filter Rows:		
	Date	COUNT(*)
▶	2003-01-01	12
	2003-01-02	12
	2003-01-03	12
	2003-01-06	12
	2003-01-07	12
	1997-01-01	7
	1997-01-02	7
	1997-01-03	7
	1997-01-06	7
	1997-01-07	7
	2001-01-01	2
	2001-01-02	2

```

18 • SELECT Date, COUNT(*)
19 FROM reliance
20 GROUP BY Date
21 HAVING COUNT(*) > 1;

```

Result Grid   Filter Rows:		
	Date	COUNT(*)
▶	1997-01-01	2
	1997-01-02	2
	1997-01-03	2
	1997-01-06	2
	1997-01-07	2

```

22 •   SELECT Date, COUNT(*)
23     FROM adanigroup
24     GROUP BY Date
25     HAVING COUNT(*) > 1;

```

Result Grid |   Filter Rows:

Date	COUNT(*)
1997-01-01	2
1997-01-02	2
1997-01-03	2
1997-01-06	2
1997-01-07	2

Result:

The result grid shows dates that appear multiple times in the dataset, with the count of how many times each date is repeated.

Identifying duplicate entries is a crucial part of data quality checks. Duplicates can lead to inaccurate reporting, so it is essential to identify and handle them either by removing or merging duplicates to ensure the dataset is clean and reliable for analysis.

STEP 3 Check for negative values in numeric fields

```
SELECT COUNT(*) AS negative_values_count
```

```
FROM table_name
```

```
WHERE open < 0 OR high < 0 OR low < 0 OR adjclose < 0 OR volume < 0;
```

Explanation:

- **SELECT COUNT(*):** This counts the total number of rows that meet the conditions in the WHERE clause.

- **AS negative_values_count:** This renames the count result to negative_values_count for clarity.
- **FROM reliance:** Specifies that the query is executed on the reliance table, which contains stock market data for Reliance.
- **WHERE clause:** This checks if any of the following columns (open, high, low, adjclose, or volume) contain values less than 0.
 - **open, high, low, adjclose, volume** are numeric columns typically used in stock market data, and none of these should have negative values under normal circumstances.
 - Using **OR** connects multiple conditions, so even if one of these columns has a negative value in a row, that row will be counted.

```

27 • SELECT
28     COUNT(*) AS negative_values_count
29 FROM
30     reliance
31 WHERE
32     open < 0 OR high < 0 OR low < 0 OR adjclose < 0 OR volume < 0;
33
34

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
negative_values_count				
0				

```

7 • SELECT
8     COUNT(*) AS negative_values_count
9 FROM
10     adanigroup
11 WHERE
12     open < 0 OR high < 0 OR low < 0 OR adjclose < 0 OR volume < 0;
13
14

```

sult Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

negative_values_count
0

```

1 • SELECT
2     COUNT(*) AS negative_values_count
3 FROM
4     tata
5 WHERE
6     open < 0 OR high < 0 OR low < 0 OR adjclose < 0 OR volume < 0;
7
8

```

sult Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

negative_values_count
0

Result:

- The result 0 indicates that none of the rows in the reliance,Adanigroups,Tata table have negative values in any of the checked columns.

This query helps ensure data quality by confirming that all the numeric fields are valid and contain no negative values where they shouldn't.

CHAPTER III

DATA INSIGHTS

TOOL USED: MY SQL WORKBENCH.

TASK1: Daily Percentage Change in Stock Prices (Close)

```
SELECT date, ((close - open) / open) * 100 AS percentage_change  
FROM table_name  
ORDER BY date;
```

This query is repeated for the other companies with slight changes to the FROM clause.

Explanation:

1. **SELECT date:** Retrieves the date column, which represents the specific trading day.
2. **((close - open) / open) * 100 AS percentage_change:**
 - **close - open:** This calculates the difference between the closing and opening prices of the stock for a given day.
 - **(close - open) / open:** This gives the percentage change relative to the opening price. It represents how much the stock price has moved up or down compared to where it started on that day.
 - *** 100:** Multiplies the result by 100 to convert the change into a percentage.
 - **AS percentage_change:** Renames the calculated column as percentage_change.
3. **FROM reliance:** Specifies the data source as the reliance table. Similarly, other tables like tata and adanigroup are queried in the following statements.
4. **ORDER BY date:** This orders the results by the date in ascending order, so you can see the daily percentage change chronologically.

```

8      /*chap 3*/
9  •    SELECT date, ((close - open) / open) * 100 AS percentage_change
10     FROM reliance
11     ORDER BY date;
12
13  •    SELECT date, ((close - open) / open) * 100 AS percentage_change
14     FROM tata
15     ORDER BY date;
16
17  •    SELECT date, ((close - open) / open) * 100 AS percentage_change
18     FROM adanigroup

```

result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

date	percentage_change
1997-01-01	-0.7363770084886282
1997-01-01	4.7255341055431925
1997-01-02	-2.6142189334096155
1997-01-02	-0.7352941176470588
1997-01-03	1.9524793759897907
1997-01-03	0.8941855257306737
1997-01-06	-1.2385273113746078
1997-01-06	-0.8823506972369025
1997-01-07	2.3391790035497877
1997-01-07	0.7903312140311786
2001-01-01	0

```

43  •    SELECT date, ((close - open) / open) * 100 AS percentage_change
44     FROM tata
45     ORDER BY date;
46
47  •    SELECT date, ((close - open) / open) * 100 AS percentage_change
48     FROM adanigroup

```

result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

date	percentage_change
1996-01-01	0.5291030417968382
1996-01-02	-0.5789441983676121
1996-01-03	-0.3308653061482771
1996-01-04	-0.756876864723111
1996-01-05	-0.36124634926798543
1997-01-01	4.700854700854707
1997-01-01	3.8592233009708647
1997-01-01	5.158728983842371
1997-01-01	0
1997-01-01	2.7777777777777777
1997-01-01	4.156079332997819

result 15 x

```

47 • SELECT date, ((close - open) / open) * 100 AS percentage_change
48 FROM adanigroup
49 ORDER BY date;

```

50

51

52

53

54

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
date	percentage_change			
2003-01-01	0.9083721065728195			
2003-01-02	0.1383380139846178			
2003-01-03	-0.7534142077750948			
2003-01-06	-1.4382800387638193			
2003-01-07	0.1389143937348096			
2008-01-01	1.3956184608439375			
2008-01-02	-0.21369875393246468			
2008-01-03	-2.2375464896132664			
2008-01-04	-3.006720107828142			
2008-01-07	-1.028227036999118			
2010-01-04	-0.2518891687657431			

TASK 2 : Find the Date with Maximum Volume

```
SELECT date, volume AS max_volume
```

```
FROM table_name
```

```
WHERE volume = (SELECT MAX(volume) FROM table_name);
```

Explanation:

1. **SELECT date, volume AS max_volume:**

- Retrieves the date on which the maximum volume was traded.
- **volume AS max_volume** renames the volume column to max_volume for clarity.

2. **FROM adanigroup:** Specifies that you're querying data from the adanigroup table (which stores the stock trading data of Adani Group).

3. **WHERE volume = (SELECT MAX(volume) FROM adanigroup):**

- This condition filters the row where the trading volume is equal to the maximum volume.
- **SELECT MAX(volume)** finds the highest volume of shares traded in the adanigroup table.
- The query compares each row's volume with this maximum value, retrieving the row where the match occurs.

```
51      /*Find the Date with Maximum Volume*/
52 •    SELECT date, volume AS max_volume
53      FROM adanigroup
54      WHERE volume = (SELECT MAX(volume) FROM adanigroup);
55
56 •    SELECT date, volume AS max_volume
57      FROM tata
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

date	max_volume
2016-01-01	10569317

```

50 • SELECT date, volume AS max_volume
51 FROM reliance
52 WHERE volume = (SELECT MAX(volume) FROM reliance);
53
54

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content

date	max_volume
1997-01-01	130531268

Result:

- The output shows the **date** (2016-01-01) when the maximum trading volume occurred, and the corresponding **volume** (10,569,317).

This query is useful for identifying peak trading days, which can indicate high activity or interest in the stock on that specific day.

TASK 3 : Price Volatility (Difference Between High and Low Prices)

Explanation:

1. SELECT:

This part of the query selects two columns:

- date: The date of the stock data.
- (high - low) AS price_volatility: This calculates the difference between the highest (high) and lowest (low) prices for the stock on that particular day. This difference is called **price volatility**, as it measures how much the price fluctuated within that day.

2. FROM

This specifies the source table, which is named reliance. It contains the stock data for the company Reliance, including date, high, and low prices.

3. ORDERBY

This ensures that the results are sorted by the date, displaying the price volatility in chronological order.

```

65
66 • SELECT date, (high - low) AS price_volatility
67 FROM reliance
68 ORDER BY date;
69
70 • SELECT date, (high - low) AS price_volatility
71 FROM tata
72 ORDER BY date;
73
74 • SELECT date, (high - low) AS price_volatility
75 FROM adanigroup

```

Result Grid |  Filter Rows: | Export:  | Wrap Cell C

date	price_volatility
1997-01-01	2.549999237060547
1997-01-01	0.8509941101074219
1997-01-02	0.49512290954589844
1997-01-02	1.3499984741210938
1997-01-03	0.3636054992675781
1997-01-03	1.25
1997-01-06	0.3481330871582031
1997-01-06	1
1997-01-07	3
1997-01-07	0.3597393035888672
2001-01-01	0

Result 9 ✕

```

70 • SELECT date, (high - low) AS price_volatility
71 FROM tata
72 ORDER BY date;
73
74 • SELECT date, (high - low) AS price_volatility
75 FROM adanigroup

```

Result Grid |  Filter Rows: | Export:  | Wrap Cell C

date	price_volatility
1996-01-01	0.47907299999999964
1996-01-02	1.51387800000000054
1996-01-03	0.71861300000000048
1996-01-04	1.05397100000000042
1996-01-05	0.7281959999999997
1997-01-01	1.10000000000000014
1997-01-01	0.94499899999999993
1997-01-01	0.63208599999999993
1997-01-01	0

```

74 • SELECT date, (high - low) AS price_volatility
75 FROM adanigroup
76 ORDER BY date;
77
78
79
80
81
82
83
84

```

Result Grid  Filter Rows: Export:  Wrap Cell C		
	date	price_volatility
▶	2003-01-01	0.01839292049407959
	2003-01-02	0.00613093376159668
	2003-01-03	0.021458983421325684
	2003-01-06	0.021458029747009277
	2003-01-07	0.0168609619140625
	2008-01-01	11.059982299804688
	2008-01-02	10.79998779296875
	2008-01-03	10.710006713867188
	2008-01-04	12.3599853515625
	2008-01-07	12.779998779296875

Result:

The result is a table showing date and price_volatility, sorted by date, to analyze the daily price fluctuations of Reliance's stock.

TASK 4 : Top 5 Dates with Highest Stock Closing Price

Explanation:

1. **Selecting Columns:** The query selects two columns, date and close, which likely represent the date and the closing stock price for each day.
2. **Sorting:** The ORDER BY close DESC part sorts the results in descending order based on the close values, showing the highest prices at the top.
3. **Limiting Results:** The LIMIT 5 clause restricts the output to only the top 5 records, ensuring that you only see the highest 5 closing prices in the dataset.

```
80      / Top 5 Dates with Highest Stock Close
81 •    SELECT date, close
82      FROM tata
83      ORDER BY close DESC
84      LIMIT 5;
85
86
87
```

Result Grid			Filter Rows:	Export
	date	close		
	2011-01-03	566.849976		
	2011-01-04	557.549988		
	2011-01-06	553.450012		
	2011-01-05	548.099976		
	2011-01-07	540		

```

91 • SELECT date, close
92 FROM adanigroup
93 ORDER BY close DESC
94 LIMIT 5;
95

```

Result Grid  Filter Rows:		
	date	close
▶	2022-09-07	698.35
	2022-09-06	691.5
	2022-09-01	683.3
	2022-09-02	676.4
	2022-09-05	670.85

```

86 • SELECT date, close
87 FROM reliance
88 ORDER BY close DESC
89 LIMIT 5;
90

```

Result Grid  Filter Rows: Exp		
	date	close
▶	2014-01-03	1598.8
	2014-01-06	1510
	2014-01-02	1490.45
	2014-01-07	1435.55
	2014-01-01	1417.4

TASK 5 : Cumulative Return Over Time

Explanation

1. **Selecting Columns:** The query selects two columns: date and a calculated cumulative_return.
2. **Calculating Cumulative Return:**

- The formula $((\text{close} / (\text{SELECT MIN}(\text{close}) \text{ FROM tata})) - 1) * 100$ is used to calculate the cumulative return based on the stock's closing price (close) relative to the minimum closing price in the dataset.
- $(\text{SELECT MIN}(\text{close}) \text{ FROM tata})$ finds the minimum closing price in the "tata" table, setting a baseline for the calculation.
- For each date, $\text{close} / \text{MIN}(\text{close})$ calculates the growth factor, subtracting 1 and multiplying by 100 to express it as a percentage.

3. **Ordering Results:** The ORDER BY date arranges the output by date, allowing you to see the cumulative return progression over time.

```

98 • SELECT date, ((close / (SELECT MIN(close) FROM tata)) - 1) * 100 AS cumulative_return
99 FROM tata
100 ORDER BY date;
101

```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	date	cumulative_return
▶	1996-01-01	15393.537446808512
	1996-01-02	15303.838510638301
	1996-01-03	15252.872553191492
	1996-01-04	15136.670212765961
	1996-01-05	15081.62829787234
	1997-01-01	5112.765957446809
	1997-01-01	4452.127659574468
	1997-01-01	2620.527021276596
	1997-01-01	15077.305531914893
	1997-01-01	884.0425531914894

```

101
102 • SELECT date, ((close / (SELECT MIN(close) FROM tata)) - 1) * 100 AS cumulative_return
103 FROM reliance
104 ORDER BY date;
105
106 • SELECT date, ((close / (SELECT MIN(close) FROM tata)) - 1) * 100 AS cumulative_return
107 FROM adanigroup
108 ORDER BY date;
109

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
date	cumulative_return		
1997-01-01	7070.212928284991		
1997-01-01	3511.3739013671875		
1997-01-02	3425.7810227414398		
1997-01-02	7080.851063829788		
1997-01-03	3466.931460766082		
1997-01-03	7102.127334919382		
1997-01-06	3443.8870369119845		
1997-01-06	7070.212928284991		
1997-01-07	7346.808510638299		
1997-01-07	3468.5774620543134		

```

102
106 • SELECT date, ((close / (SELECT MIN(close) FROM tata)) - 1) * 100 AS cumulative_return
107 FROM adanigroup
108 ORDER BY date;
109

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
date	cumulative_return		
2003-01-01	135.45638043829737		
2003-01-02	136.10873425260505		
2003-01-03	136.27169588778884		
2003-01-06	134.64126789823493		
2003-01-07	135.13043180425117		
2008-01-01	55085.10534408245		
2008-01-02	55536.16813497341		
2008-01-03	54189.362481299875		
2008-01-04	52680.8526221742		
2008-01-07	52123.40360600898		

Result

The cumulative_return column shows the percentage increase in the stock price from the lowest recorded closing price to each date in the dataset. This is useful for understanding the stock's performance over time and tracking long-term growth.

CHAPTER IV

LEARNING OUTCOME, CHALLENGES FACED, RECOMMENDATION.

Learning Outcomes:

1. Data Querying and Analysis with SQL:

SQL was used to query the datasets for analysing stock data across companies like Tata, Reliance, and Adani. This helped in retrieving, manipulating, and filtering data based on key attributes such as `Date`, `Open`, `Close`, `High`, `Low`, and `Volume`. SQL proved effective in extracting meaningful insights by applying conditions and performing calculations like percentage changes and cumulative returns.

2. Aggregation and Grouping:

The queries involving `GROUP BY` and `HAVING` clauses allowed for the identification of trends in grouped data, such as detecting duplicate dates or aggregating the trading volume. Understanding how to group data based on specific columns enabled insights into how stocks behaved on particular dates, as well as identifying days with maximum trading activity.

3. Data Transformation with SQL:

SQL functions, such as `STR\_TO\_DATE`, were used to transform and standardize the date format within the datasets, ensuring consistency and enabling further analysis. This showcased the ability of SQL to handle data cleaning tasks, including converting and restructuring data.

4. Calculating Financial Metrics with SQL:

Various financial metrics, such as “percentage change” and “price volatility”, were calculated directly within SQL queries. This demonstrated how SQL can be leveraged to perform essential stock market calculations without needing external tools or manual data manipulation.

5. Optimization of Queries:

SQL queries such as those finding maximum values or top records (e.g., dates with the highest closing prices) emphasized the importance of optimization. Efficient querying ensured the quick retrieval of data, even for large datasets like stock market records.

Challenges Faced:

1. Handling Duplicate Dates:

Identifying and handling duplicate `Date` values using SQL's `GROUP BY` and `HAVING` was a challenge. Duplicate entries can skew analysis, so the ability to detect and remove them through queries was essential.

2. Date Format Conversion:

Converting the date format using `STR\_TO\_DATE` posed a challenge, especially when dealing with inconsistencies in date formats across datasets. SQL had to be carefully used to ensure proper formatting, which could otherwise affect time-series analysis.

3. Complex Calculations in SQL:

Performing calculations like percentage change and cumulative return required a deep understanding of SQL's arithmetic capabilities. Complex mathematical operations had to be integrated directly into the queries, ensuring the results matched expectations without errors.

4. Efficient Data Retrieval:

Queries to retrieve top records (e.g., top 5 dates with the highest closing price) required optimization for efficient execution. Large datasets can slow down the query process, especially when ordering and filtering data, so careful structuring of SQL queries was needed for optimal performance.

Recommendations:

1. Improving Query Efficiency:

For large datasets, it's recommended to create indexes on frequently queried columns, such as 'Date', 'Volume', and 'Close', to enhance the performance of queries, especially those involving 'ORDER BY' or 'GROUP BY'. Indexes reduce the time it takes for SQL to retrieve and organize data.

2. Data Consistency Checks:

Before running analyses, it is advisable to implement SQL queries that ensure data consistency (e.g., checking for duplicate or null values). Running validation queries in advance will help ensure cleaner data, leading to more accurate results.

3. Exploring SQL Joins:

For future work involving multiple datasets, exploring SQL joins can allow for merging data across companies (e.g., Tata, Reliance, and Adani) and performing cross-company analysis. This will provide a more holistic view of stock performance across the market.

This SQL-focused approach highlights how powerful query operations can provide deep insights into stock market data, offering a clear view of stock performance, trends, and volatility over time.